tidymodels pipelines

Workflow labs use the variant template: Goal → Approach → Execution → Check → Report.

Learning objectives

- Build a `tidymodels` recipe, workflow, and tuning grid for a binary classification task.
- Split data for CV properly and collect metrics across resamples.
- Finalise a model on the full training data and evaluate on a test split.

Prerequisites

Cross-validation and regularisation.

Background

`tidymodels` organises the modelling workflow into composable objects: a *recipe* describes preprocessing, a *model specification* describes the algorithm and its tunable parameters, a *workflow* bundles them, and *tune* functions drive resampling. The value of this separation is reproducibility: the same recipe runs on training, tuning, and test data, and it is hard to leak information across the split.

This lab builds a full pipeline on `MASS::Pima.tr`: logistic regression with elastic net and tuned `mixture` and `penalty`. The same pattern extends to random forests, boosting, neural networks, or any model with a parsnip wrapper.

The convention to remember: tune on the training folds, collect metrics, finalise with the best parameters, fit once to all of the training data, then evaluate once on the test set. The test set is touched exactly once.

Setup

```
library(tidyverse)
library(tidymodels)
library(MASS)
set.seed(42)
theme_set(theme_minimal(base_size = 12))
```

1. Goal

Fit an elastic-net logistic regression to predict diabetes status on `Pima.tr`, using a proper tidymodels pipeline.

2. Approach

```
ggplot(d, aes(glu, bmi, colour = type)) + geom_point(alpha = 0.7)
```

3. Execution

Split, recipe, model, workflow, tune.

```
d_train <- training(d_split); d_test <- testing(d_split)
folds <- vfold_cv(d_train, v = 5, strata = type)

rec <- recipe(type ~ ., data = d_train) |>
  step_normalize(all_numeric_predictors())

mod <- logistic_reg(penalty = tune(), mixture = tune()) |>
  set_engine("glmnet")

wf  <- workflow() |> add_recipe(rec) |> add_model(mod)

grid <- grid_regular(penalty(), mixture(), levels = 5)
res  <- tune_grid(wf, resamples = folds, grid = grid,
                  metrics = metric_set(roc_auc, accuracy))
collect_metrics(res) |>
  filter(.metric == "roc_auc") |>
  arrange(desc(mean)) |>
  head(5)
```

4. Check

Select best, finalise, evaluate on the test set.

```
wf_final <- finalize_workflow(wf, best)
fit_final <- fit(wf_final, data = d_train)
preds <- predict(fit_final, d_test, type = "prob") |>
  bind_cols(predict(fit_final, d_test), d_test)
roc_auc(preds, truth = type, .pred_Yes, event_level = "second")
accuracy(preds, truth = type, estimate = .pred_class)
```

```
  autoplot()
```

5. Report

> An elastic-net logistic regression tuned on 5-fold CV, using a 5×5 grid over `penalty` and `mixture`, achieved a test ROC-AUC of `round(roc_auc(preds, truth = type, .pred_Yes, event_level = "second")$.estimate, 3)` on `MASS::Pima.tr`.

The pipeline pattern — recipe + workflow + tune + finalise — is the template we will reuse for every predictive-model lab hereafter.

Common pitfalls

- Normalising the entire dataset before splitting; leakage is then baked in.
- Reporting the mean CV metric as if it were a test-set metric.
- Tuning without stratifying folds on an imbalanced outcome.

Further reading

- Kuhn M, Silge J, *Tidy Modeling with R* (online book).

Session info

```
```

See also — chapter index

- Methods in this chapter
- Find your method (Chapter 0)